DAY 1 OF 7

Project setup & database models

A running Flask backend with every table and an admin user created automatically

Today's plan

10 min	Bootcamp overview, demo of the finished app
15 min	Architecture: API backend + Vue frontend
15 min	Setup: Python venv, folders, requirements
20 min	RBAC: roles with Flask-Security-Too
30 min	Live code: config, extensions, models
15 min	Live code: seed admin, run the app
15 min	Recap and questions

By the end you can

- Explain how MAD II differs from MAD I
- Structure a Flask project with an app factory
- Design the Quiz Master database with SQLAlchemy
- Set up role-based access (RBAC) with Flask-Security-Too

MAD I vs MAD II

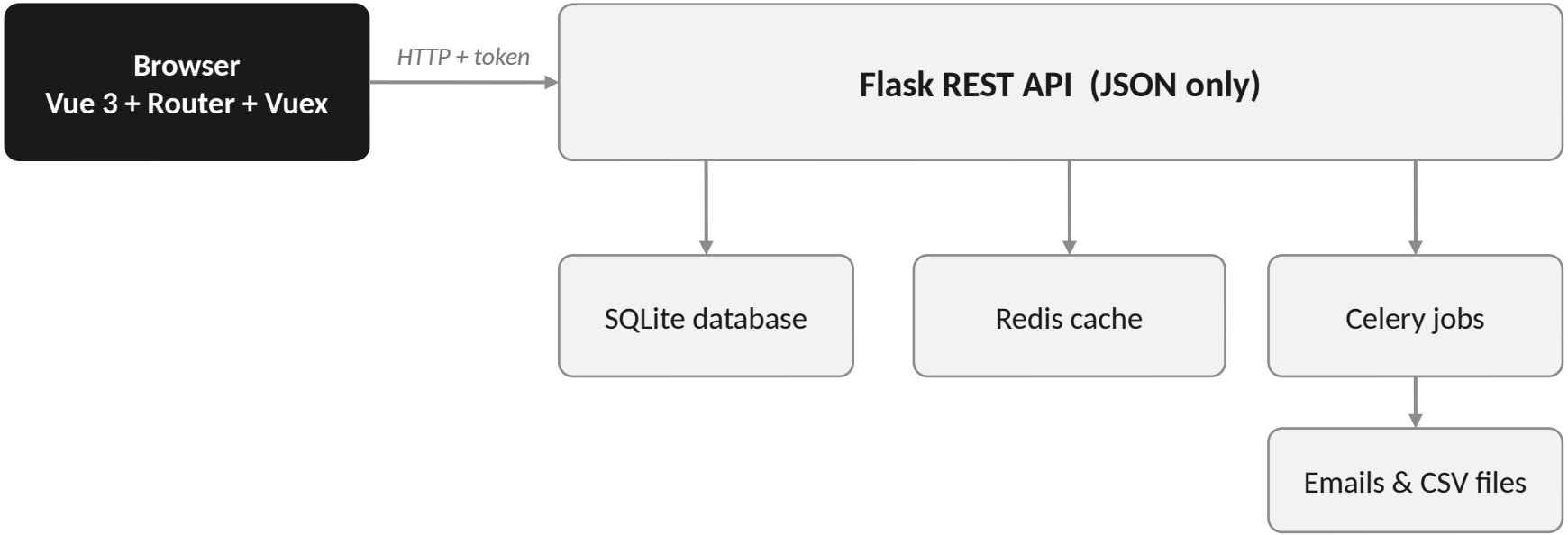
MAD I

- Flask renders HTML pages with Jinja
- Every click reloads the whole page
- One server does everything

MAD II

- Flask returns JSON only (an API)
- Vue draws the pages in the browser
- Two servers: API on :5000, UI on :5173

The app in one picture



Today we build the Flask + SQLite part. Vue arrives on Day 4, Celery on Day 6, Redis caching on Day 7.

Three ideas behind the project structure

App factory

create_app() builds the app in one place: config, extensions, blueprints. No circular imports.

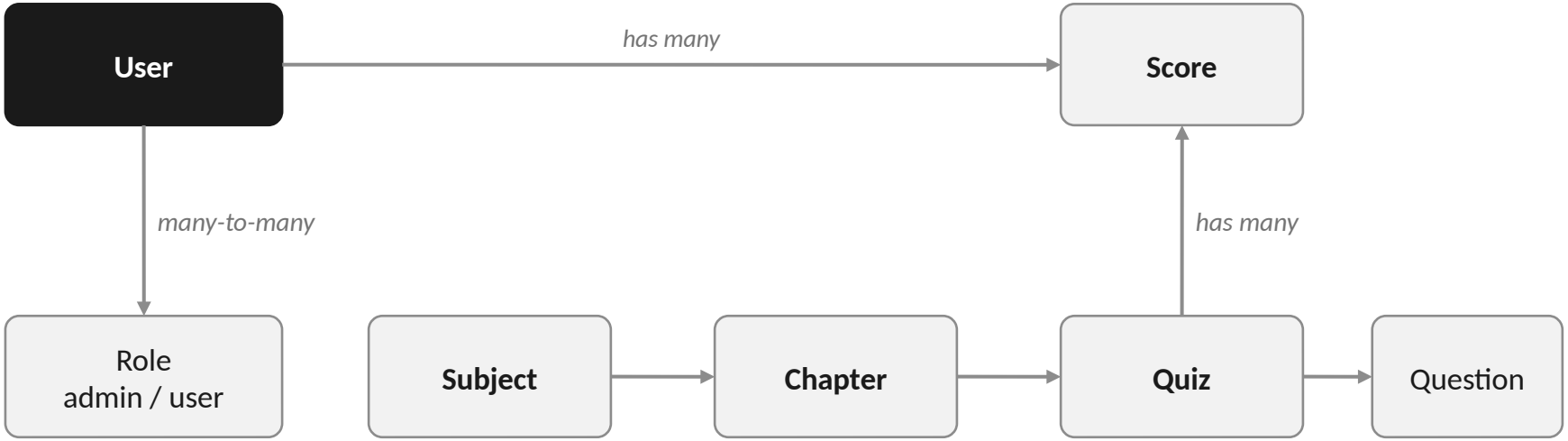
extensions.py

db = SQLAlchemy() is created without an app and connected later with db.init_app(app).

Flask-Security-Too

Gives password hashing, roles and tokens. Today we only use its User and Role models.

Database design



Each arrow points from one to many. cascade="all, delete-orphan": deleting a subject also deletes its chapters, quizzes, questions and scores.

Tables and their main columns

Table	Columns
User	email, password (hashed), full_name, qualification, dob, active, fs_uniquifier, last_visit
Role	name (admin / user)
Subject	name, description
Chapter	subject_id, name, description
Quiz	chapter_id, title, date_of_quiz, time_duration (minutes), remarks, created_at
Question	quiz_id, statement, option1 … option4, correct_option (1–4)
Score	quiz_id, user_id, time_stamp_of_attempt, total_scored, total_questions

fs_uniquifier is required by Flask-Security-Too; changing it logs the user out everywhere (Day 2).

What the user table looks like

`table` **`user`**

id	email	password	full_name	qualification	dob	active	fs_uniquifier	last_visit
PK	*unique*	*bcrypt hash*	*required*	*optional*	*date*	*bool*	*unique*	*datetime*
1	admin@quizmaster.com	$2b$12$…	Quiz Master			1	9f2c…e1	2026-10-20
2	user@quizmaster.com	$2b$12$…	Demo User	B.Sc	2001-05-14	1	41ab…7d	2026-10-20
3	ann@example.com	$2b$12$…	Ann Mathew	B.Tech	2002-11-02	0	c03e…5a	2026-10-18

The password column never holds the real password, only a bcrypt hash. active = 0 means the admin blocked Ann. created_at and the time part of last_visit are not shown to save space.

What the role table looks like

`table` **`role`**

id	name	description
PK	*unique*	*optional*
1	admin	Quiz Master (superuser)
2	user	Quiz taker

Only two rows, created by seed.py on start. Nobody can sign up as admin: register always gives the user role.

What the roles_users table looks like

`table` **`roles_users`**

user_id	role_id	meaning (not a column)
FK → user.id	*FK → role.id*	
1	1	admin@quizmaster.com is an admin
2	2	user@quizmaster.com is a user
3	2	ann@example.com is a user

A link table with only two ids per row. It lets one user have many roles and one role belong to many users (many-to-many).

JOIN: which role does each user have?

SQL

```
SELECT user.email, role.name
FROM user
JOIN roles_users
  ON roles_users.user_id = user.id
JOIN role
  ON role.id = roles_users.role_id;
```

email	role
admin@quizmaster.com	admin
user@quizmaster.com	user
ann@example.com	user

SQLAlchemy (flask shell)

```
# the relationship does the join
admin = db.session.get(User, 1)
[r.name for r in admin.roles]
# ['admin']

# or write the join yourself
db.session.query(User.email, Role.name)\
  .join(User.roles).all()
# [('admin@quizmaster.com', 'admin'), ...]

# Flask-Security shortcut (Day 2)
admin.has_role("admin")     # True
```

Both sides do the same thing: user → roles_users (same user_id) → role (same role_id). secondary=roles_users in the model is why SQLAlchemy can do it for us.

What the subject table looks like

`table` **`subject`**

id	name	description
PK	*unique, required*	*optional*
1	Python	All about Python
2	Web Development	All about Web Development

The top of the tree. Deleting Python deletes its chapters, their quizzes, questions and scores (cascade).

What the chapter table looks like

`table` **`chapter`**

id	subject_id	name	description
PK	*FK → subject.id*	*required*	*optional*
1	1	Basics	Basics chapter
2	1	Data Structures	Data Structures chapter
3	2	HTTP	HTTP chapter
4	2	Vue.js	Vue.js chapter

subject_id says which subject a chapter belongs to: chapters 1–2 are Python, 3–4 are Web Development.

What the quiz table looks like

`table` **quiz**

id	chapter_id	title	date_of_quiz	time_duration	remarks
PK	*FK → chapter.id*	*required*	*date*	*minutes*	*optional*
1	1	Basics Quiz 1	2026-10-20	5	
2	2	Data Structures Quiz 1	2026-10-21	5	
3	3	HTTP Quiz 1	2026-10-21	5	Open book

time_duration drives the countdown timer on Day 5. created_at is filled automatically and used by the daily reminder on Day 6.

What the question table looks like

`table` **question**

id	quiz_id	statement	option1	option2	option3	option4	correct_option
PK	*FK → quiz.id*	*required*	*required*	*required*	*optional*	*optional*	*1 to 4*
1	1	Output of print(2 ** 3)?	6	8	9	5	2
2	1	Keyword for a function?	func	def	lambda	fn	2
3	1	Which type is immutable?	list	dict	tuple	set	3

correct_option stores the number of the right option, not its text. This column is never sent to the quiz taker (Day 3).

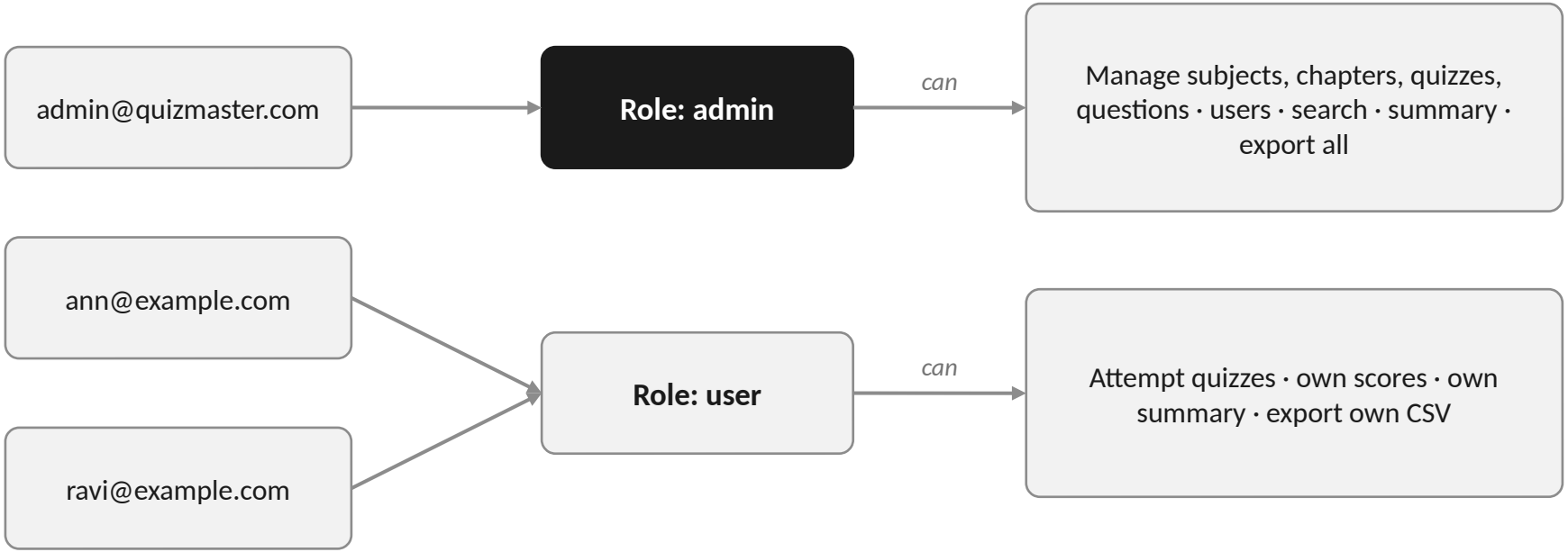
What the score table looks like

`table` **`score`**

id	quiz_id	user_id	time_stamp_of_attempt	total_scored	total_questions
PK	*FK → quiz.id*	*FK → user.id*	*datetime*	*correct answers*	*questions in quiz*
1	1	2	2026-10-20 10:02	2	3
2	3	2	2026-10-20 10:15	2	2
3	1	3	2026-10-18 18:20	3	3

One row per attempt, so a user can retake a quiz. Percentage = total_scored / total_questions. The summary charts on Day 7 are built from this table.

RBAC: the role decides what you can do



Users are not given permissions one by one. Each user gets a role; every API checks the role. Users ⟷ roles is many-to-many (roles_users table).

Roles in the database

backend/application/models.py · app.py

```
roles_users = db.Table("roles_users",
    db.Column("user_id", db.Integer, db.ForeignKey("user.id")),
    db.Column("role_id", db.Integer, db.ForeignKey("role.id")))

class Role(db.Model, RoleMixin):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(80), unique=True)

class User(db.Model, UserMixin):
    ...
    roles = db.relationship("Role", secondary=roles_users,
                            backref=db.backref("users"))

datastore = SQLAlchemyUserDatastore(db, User, Role)
security.init_app(app, datastore)
```



roles_users

link table: many users, many roles



Mixins

RoleMixin / UserMixin add has_role() and more



Datastore

tells Flask-Security which models to use

Giving a user a role

backend/application/seed.py · api/auth.py

```
# seed.py – roles exist before anyone signs up
datastore.find_or_create_role(name="admin")
datastore.find_or_create_role(name="user")
# the one and only admin
datastore.create_user(email=ADMIN_EMAIL,
                      password=hash_password(ADMIN_PASSWORD),
                      roles=["admin"])

# auth.py – every new sign-up is a normal user
datastore.create_user(email=email,
                      password=hash_password(password),
                      roles=["user"])

# change it later if needed
datastore.add_role_to_user(user, "admin")
datastore.remove_role_from_user(user, "admin")
```



Admin by code

nobody can register as admin



Default role

register always gives "user"



Change roles

add / remove with the datastore

Checking the role

backend/application/api/*.py

```
@app.get("/api/admin/ping")
@auth_required("token")            # 401 if no valid token
@roles_required("admin")           # 403 if not admin
def admin_ping(): ...

@app.get("/api/profile")           # example
@auth_required("token")
@roles_accepted("admin", "user")   # any ONE of these roles
def profile(): ...

# inside a route: different data per role
if current_user.has_role("admin"):
    quizzes = Quiz.query.all()
else:
    quizzes = [q for q in Quiz.query.all() if q.questions]
```



roles_required

user must have ALL listed roles



roles_accepted

user needs ANY one of them



has_role()

branch inside the route

Who can call what

API group	Admin	User	No token
/api/register, /api/login	yes	yes	yes
GET subjects, chapters, quizzes	yes	yes	401
POST / PUT / DELETE subjects … questions	yes	403	401
/api/quizzes/<id>/attempt, /submit	403	yes	401
/api/scores, /api/summary, /api/export	yes	yes	401
/api/admin/* (users, search, summary, jobs)	yes	403	401

Vue also hides admin pages from users (Day 4), but that is only for convenience – the backend role check is the real protection.

A model in SQLAlchemy

backend/application/models.py

```
class Quiz(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    chapter_id = db.Column(db.Integer,
        db.ForeignKey("chapter.id"), nullable=False)
    title = db.Column(db.String(120), nullable=False)
    date_of_quiz = db.Column(db.Date, default=date.today)
    time_duration = db.Column(db.Integer, default=10)
    remarks = db.Column(db.Text)
    questions = db.relationship("Question", backref="quiz",
                    cascade="all, delete-orphan")
```



Foreign key

chapter_id links each quiz to its chapter



relationship

quiz.questions gives the list; question.quiz goes back



cascade

delete a quiz → its questions go too

The admin is created by code, never registered

backend/application/seed.py

```
def seed_database():
    datastore = current_app.extensions["security"].datastore
    db.create_all()
    datastore.find_or_create_role(name="admin")
    datastore.find_or_create_role(name="user")
    db.session.commit()

    if not datastore.find_user(email=ADMIN_EMAIL):
        datastore.create_user(
            email=ADMIN_EMAIL,
            password=hash_password(ADMIN_PASSWORD),
            full_name="Quiz Master",
            roles=["admin"])
        db.session.commit()
```



create_all()

creates missing tables on every start



Two roles

admin and user exist before anyone logs in



hash_password

passwords are never stored as plain text

Run and check

Terminal

```
cd quiz-master/backend
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py

# open in the browser:
http://localhost:5000/api/health
```

Check that...

- /api/health returns {"status": "ok"}
- instance/quiz_master.db file is created
- The DB has 7 tables and one admin row
- The admin password is a long hash, not "admin123"

Common errors

Error	Fix
`ModuleNotFoundError: flask_security`	venv not activated, or pip install not run
`ValueError … fs_uniquifier`	User model is missing the fs_uniquifier column
`Changed a model, table did not change`	create_all() never alters tables – delete instance/quiz_master.db and run again
`Working outside of application context`	run DB code inside "with app.app_context():"

Recap

- Flask now serves JSON only; Vue will draw the pages
- create_app() + extensions.py keep setup in one place
- Seven tables model subjects → chapters → quizzes → questions, users and scores
- RBAC: every user has a role (admin or user); the role decides which APIs they may call
- The admin and both roles are created automatically on start

Next: Day 2 – register, login and protect every API with tokens and roles.